DESIGN AND ANALYSIS OF ALGORITHMS

DAY 1 – ALGORITHM ANALYSIS FUNDAMENTALS

Name: N. PARNITA

Register No.: 192525322

Course code: CSA0617

Course Name: Design and Analysis of Algorithm

TABLE OF CONTENTS

Experiment 1: Linear Search (Iterative)

Objective: To analyze the time complexity of linear search.

Problem Statement: Write a Python program to search for a key in an array using linear search.

Sample Input: [10, 25, 30, 45, 50], Key = 30

Sample Output: Key found at index 2

Time Complexity: Best Case: O(1) | Worst Case: O(n) | Average Case: Θ(n)

Explanation: Linear search checks each element sequentially until the required key is found. It is simple, but the number of comparisons can increase linearly with the input size.

Python Program

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

arr = list(map(int, input("Enter elements: ").split()))
key = int(input("Enter key: "))

index = linear_search(arr, key)

if index != -1:
    print("Key found at index", index)
else:
    print("Key not found")
```

Execution / Output

```
Enter elements: 10 25 30 45 50
Enter key: 30
Key found at index 2
```

Result: The program was implemented and the expected result was obtained.

Experiment 2: Binary Search (Recursive)

Objective: To analyze the complexity of recursive binary search.

Problem Statement: Implement recursive binary search in Python.

Sample Input: [5, 10, 15, 20, 25], Key = 20

Sample Output: Key found at index 3

Time Complexity: Best Case: O(1) | Worst Case: O(log n) | Average Case: Θ(log n)

Explanation: Binary search repeatedly divides the sorted search interval into two halves. This divide-and-conquer approach gives logarithmic search time.

Python Program

```
def binary_search(arr, low, high, key):
    if low > high:
        return -1

    mid = (low + high) // 2

    if arr[mid] == key:
        return mid
    elif key < arr[mid]:
        return binary_search(arr, low, mid - 1, key)
    else:
        return binary_search(arr, mid + 1, high, key)

arr = list(map(int, input("Enter sorted elements: ").split()))
key = int(input("Enter key: "))

index = binary_search(arr, 0, len(arr) - 1, key)

if index != -1:
    print("Key found at index", index)
else:
    print("Key not found")
```

Execution / Output

```
Enter sorted elements: 5 10 15 20 25
Enter key: 20
Key found at index 3
```

Result: The program was implemented and the expected result was obtained.

Experiment 3: Factorial (Iterative vs Recursive)

Objective: To compare iterative and recursive factorial computation.

Problem Statement: Write programs to compute factorial using both iterative and recursive methods.

Sample Input: n = 5

Sample Output: Iterative factorial = 120
Recursive factorial = 120

Time Complexity: Both methods: O(n) time. Recursive method additionally uses O(n) call-stack space.

Explanation: The experiment compares a loop-based solution with recursive calls. Both perform a linear number of operations, while recursion uses additional stack space.

Python Program

```
def factorial_iterative(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

def factorial_recursive(n):
    if n <= 1:
        return 1
    return n * factorial_recursive(n - 1)

n = int(input("Enter n: "))

print("Iterative factorial =", factorial_iterative(n))
print("Recursive factorial =", factorial_recursive(n))
```

Execution / Output

```
Enter n: 5
Iterative factorial = 120
Recursive factorial = 120
```

Result: The program was implemented and the expected result was obtained.

Experiment 4: Fibonacci Series

Objective: To analyze recursive versus iterative Fibonacci generation.

Problem Statement: Generate the first n Fibonacci numbers.

Sample Input: n = 6

Sample Output: [0, 1, 1, 2, 3, 5]

Time Complexity: Iterative: O(n) | Naïve Recursive: O(2^n) | Memoized Recursive: O(n)

Explanation: The iterative approach generates Fibonacci numbers efficiently in linear time. Naïve recursion repeatedly recomputes the same subproblems, resulting in exponential growth.

Python Program

```
def fibonacci_iterative(n):
    a, b = 0, 1
    series = []
    for _ in range(n):
        series.append(a)
        a, b = b, a + b
    return series

n = int(input("Enter n: "))
print(fibonacci_iterative(n))
```

Execution / Output

```
Enter n: 6
[0, 1, 1, 2, 3, 5]
```

Result: The program was implemented and the expected result was obtained.

Experiment 5: Matrix Multiplication

Objective: To analyze the time complexity of nested loops in matrix multiplication.

Problem Statement: Multiply two matrices using nested loops.

Sample Input: A = [[1,2],[3,4]], B = [[5,6],[7,8]]

Sample Output: [[19, 22], [43, 50]]

Time Complexity: O(n^3) for two n × n matrices.

Explanation: Three nested loops are used to compute each element of the result matrix. Therefore, the number of operations grows cubically with matrix size.

Python Program

```
def multiply(A, B):
    rows = len(A)
    cols = len(B[0])
    common = len(B)
    C = [[0] * cols for _ in range(rows)]

    for i in range(rows):
        for j in range(cols):
            for k in range(common):
                C[i][j] += A[i][k] * B[k][j]
    return C

A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]

print(multiply(A, B))
```

Execution / Output

```
[[19, 22], [43, 50]]
```

Result: The program was implemented and the expected result was obtained.

Experiment 6: Merge Sort

Objective: To analyze divide-and-conquer sorting.

Problem Statement: Implement merge sort in Python.

Sample Input: [38, 27, 43, 3, 9, 82, 10]

Sample Output: [3, 9, 10, 27, 38, 43, 82]

Time Complexity: Best, Average and Worst Case: O(n log n).

Explanation: Merge sort divides the array into smaller halves, sorts them recursively, and merges the sorted halves. Its running time remains O(n log n).

Python Program

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

arr = list(map(int, input("Enter elements: ").split()))
print(merge_sort(arr))
```

Execution / Output

```
Enter elements: 38 27 43 3 9 82 10
[3, 9, 10, 27, 38, 43, 82]
```

Result: The program was implemented and the expected result was obtained.

Experiment 7: Quick Sort

Objective: To analyze recursive quick sort.

Problem Statement: Implement quick sort in Python.

Sample Input: [10, 7, 8, 9, 1, 5]

Sample Output: [1, 5, 7, 8, 9, 10]

Time Complexity: Best/Average: O(n log n) | Worst: O(n^2)

Explanation: Quick sort partitions the array around a pivot and recursively sorts the resulting partitions. Performance depends strongly on pivot selection.

Python Program

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[-1]
    left = [x for x in arr[:-1] if x <= pivot]
    right = [x for x in arr[:-1] if x > pivot]

    return quick_sort(left) + [pivot] + quick_sort(right)

arr = list(map(int, input("Enter elements: ").split()))
print(quick_sort(arr))
```

Execution / Output

```
Enter elements: 10 7 8 9 1 5
[1, 5, 7, 8, 9, 10]
```

Result: The program was implemented and the expected result was obtained.

Experiment 8: Tower of Hanoi

Objective: To analyze exponential recursion.

Problem Statement: Solve the Tower of Hanoi problem for n disks.

Sample Input: n = 3

Sample Output: Moves:
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C

Time Complexity: O(2^n) time.

Explanation: Tower of Hanoi uses recursive calls to move n-1 disks before and after moving the largest disk. The number of moves is 2^n - 1.

Python Program

```
def tower_of_hanoi(n, source, auxiliary, destination):
    if n == 1:
        print(f"Move disk 1 from {source} to {destination}")
        return

    tower_of_hanoi(n - 1, source, destination, auxiliary)
    print(f"Move disk {n} from {source} to {destination}")
    tower_of_hanoi(n - 1, auxiliary, source, destination)

n = int(input("Enter number of disks: "))
tower_of_hanoi(n, "A", "B", "C")
```

Execution / Output

```
Enter number of disks: 3
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

Result: The program was implemented and the expected result was obtained.

Experiment 9: GCD (Euclidean Algorithm)

Objective: To analyze recursive computation of GCD.

Problem Statement: Implement the Euclidean algorithm to find the GCD of two numbers.

Sample Input: a = 48, b = 18

Sample Output: 6

Time Complexity: O(log n) time.

Explanation: The Euclidean algorithm repeatedly replaces the pair of numbers with the smaller number and the remainder. This reduces the problem rapidly and gives logarithmic complexity.

Python Program

```
def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)

a = int(input("Enter a: "))
b = int(input("Enter b: "))

print("GCD =", gcd(a, b))
```

Execution / Output

```
Enter a: 48
Enter b: 18
GCD = 6
```

Result: The program was implemented and the expected result was obtained.

Experiment 10: Insertion Sort

Objective: To analyze iterative sorting and input sensitivity.

Problem Statement: Implement insertion sort in Python.

Sample Input: [12, 11, 13, 5, 6]

Sample Output: [5, 6, 11, 12, 13]

Time Complexity: Best Case: O(n) | Worst Case: O(n^2) | Average Case: Θ(n^2)

Explanation: Insertion sort builds the sorted portion one element at a time. It is efficient for nearly sorted input but can take quadratic time for reverse-ordered input.

Python Program

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

arr = list(map(int, input("Enter elements: ").split()))
insertion_sort(arr)
print(arr)
```

Execution / Output

```
Enter elements: 12 11 13 5 6
[5, 6, 11, 12, 13]
```

Result: The program was implemented and the expected result was obtained.

Experiment 11: Heap Sort

Objective: To analyze heap-based sorting.

Problem Statement: Implement heap sort in Python.

Sample Input: [4, 10, 3, 5, 1]

Sample Output: [1, 3, 4, 5, 10]

Time Complexity: O(n log n) in best, average and worst cases.

Explanation: Heap sort constructs a heap and repeatedly extracts the largest element. The heap structure provides consistent O(n log n) performance.

Python Program

```
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left
    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)

    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)

arr = list(map(int, input("Enter elements: ").split()))
heap_sort(arr)
print(arr)
```

Execution / Output

```
Enter elements: 4 10 3 5 1
[1, 3, 4, 5, 10]
```

Result: The program was implemented and the expected result was obtained.

Experiment 12: Counting Inversions

Objective: To analyze a modified merge sort.

Problem Statement: Count the number of inversions in an array.

Sample Input: [2, 4, 1, 3, 5]

Sample Output: 3

Time Complexity: O(n log n) time.

Explanation: An inversion is a pair of elements that appear in the wrong order. A modified merge procedure can count these pairs while sorting the array.

Python Program

```
def count_inversions(arr):
    if len(arr) <= 1:
        return arr, 0

    mid = len(arr) // 2
    left, inv_left = count_inversions(arr[:mid])
    right, inv_right = count_inversions(arr[mid:])

    merged = []
    i = j = inv = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            inv += len(left) - i
            j += 1

    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged, inv_left + inv_right + inv

arr = list(map(int, input("Enter elements: ").split()))
_, count = count_inversions(arr)
print("Number of inversions =", count)
```

Execution / Output

```
Enter elements: 2 4 1 3 5
Number of inversions = 3
```

Result: The program was implemented and the expected result was obtained.

Experiment 13: Maximum Subarray Sum

Objective: To compare Kadane's algorithm with divide-and-conquer.

Problem Statement: Find the maximum subarray sum.

Sample Input: [-2, -3, 4, -1, -2, 1, 5, -3]

Sample Output: 7

Time Complexity: Kadane's Algorithm: O(n) | Divide and Conquer: O(n log n)

Explanation: Kadane's algorithm scans the array once while maintaining the best sum ending at each position. It is more efficient than the divide-and-conquer approach for this problem.

Python Program

```
def max_subarray_sum(arr):
    current = best = arr[0]

    for x in arr[1:]:
        current = max(x, current + x)
        best = max(best, current)

    return best

arr = list(map(int, input("Enter elements: ").split()))
print("Maximum subarray sum =", max_subarray_sum(arr))
```

Execution / Output

```
Enter elements: -2 -3 4 -1 -2 1 5 -3
Maximum subarray sum = 7
```

Result: The program was implemented and the expected result was obtained.

Experiment 14: Exponentiation

Objective: To compare iterative power computation with recursive fast power.

Problem Statement: Compute x^n using iterative and recursive fast-power approaches.

Sample Input: x = 2, n = 10

Sample Output: 1024

Time Complexity: Iterative: O(n) | Recursive fast power: O(log n)

Explanation: Fast exponentiation reduces the number of multiplications by repeatedly squaring the base and halving the exponent.

Python Program

```
def power_iterative(x, n):
    result = 1
    for _ in range(n):
        result *= x
    return result

def fast_power(x, n):
    if n == 0:
        return 1
    half = fast_power(x, n // 2)
    if n % 2 == 0:
        return half * half
    return x * half * half

x = int(input("Enter x: "))
n = int(input("Enter n: "))

print("Iterative =", power_iterative(x, n))
print("Fast recursive =", fast_power(x, n))
```

Execution / Output

```
Enter x: 2
Enter n: 10
Iterative = 1024
Fast recursive = 1024
```

Result: The program was implemented and the expected result was obtained.

Experiment 15: Closest Pair of Points

Objective: To analyze brute-force and divide-and-conquer approaches for closest points.

Problem Statement: Find the closest pair of points in two-dimensional space.

Sample Input: [(1,2), (4,5), (7,8), (3,1)]

Sample Output: Closest pair: (1, 2) and (3, 1), Distance = $\sqrt{5} \approx 2.236$

Time Complexity: Brute Force: O(n^2) | Optimized Divide-and-Conquer: O(n log n)

Explanation: The brute-force approach compares every pair of points. The divide-and-conquer method reduces the number of comparisons and motivates optimization for large inputs.

Python Program

```
import math

def closest_pair(points):
    min_dist = float("inf")
    pair = None

    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            x1, y1 = points[i]
            x2, y2 = points[j]
            dist = math.sqrt((x1 - x2)**2 + (y1 - y2)**2)

            if dist < min_dist:
                min_dist = dist
                pair = (points[i], points[j])

    return pair, min_dist

points = [(1, 2), (4, 5), (7, 8), (3, 1)]
pair, distance = closest_pair(points)

print("Closest pair:", pair)
print("Distance =", round(distance, 3))
```

Execution / Output

```
Closest pair: ((1, 2), (3, 1))
Distance = 2.236
```

Result: The program was implemented and the expected result was obtained.